Chicken-Killing and Hands-Free Feedback: Training Behavior in BurlapCraft Using I-Learning and Emotion Detection

Zoe D. Papakipos and Jacob A. Beck

I. WHAT WE DID

Our goal for this research project was to get an agent in Minecraft to achieve a goal using user feedback, rather than giving the agent a reward or terminal function, so the user can teach the agent to do custom policies. In particular, we wanted to get this working with the user training the agent to go to a gold block in a small dungeon. Once we got this working, we extended the project to teaching the agent to kill a chicken in a dungeon. We achieved both of these goals using a "good" button and a "bad" button to provide user feedback in two different dungeons (one 9x9 blocks, one 3x3).

An additional task we attempted in the 9x9 dungeon was to have the agent explore away from a starting position with full hunger and get back to refuel its hunger before starving. The hope was to teach a more or less risky policy of exploration. This was also done using a "good" button and a "bad" button.

We also incorporated computer vision into the task of getting the agent to a gold block by using OpenCV's face classifiers to classify the valence of emotion on the users face as they watch the agent learn. When the user looks happy, the agent receives positive feedback. When the user looks angry, the agent receives negative feedback. We got this to work fairly well, although the accuracy is limited by the precision of OpenCV's algorithms. As computer vision and emotion detection algorithms improve, our learning methods will only improve in efficacy.

Fig. 1. Agent killing a chicken in the 3x3 dungeon

II. HOW WE DID IT

We used a learning algorithm called I-Learning, developed by James McGlashan in Stefanie's robotics lab, which collects user feedback in realtime to help an agent learn the best policy given an environment and a state-action space. When deciding what to do next, the algorithm uses the feedback given to recent actions and actions in similar states to extrapolate to the next action. This is good for our purposes because we wanted to use realtime feedback, and have the agent keep doing something we deem good (e.g. moving in a certain direction, or moving toward a target), but also want to be able to tell the agent to stop doing something in realtime (hence the "bad" button to cancel out the good feedback it may have been getting).

We used tile coding (or CMAC) to extrapolate limited feedback to other states and actions, allowing us to have the agent guess increasingly well without trying every state-action combination. The variables we tiled on were different for the gold block goal, killing the chicken, and hunger exploration. For getting to the gold block, we tiled on the agent's x, z, and the direction it was facing. For killing the chicken, we tiled on the agents relative x and z (i.e. agent_x - chicken_x and agent_z - chicken_z) and the agents direction. For hunger exploration, we tiled on the agent's x, z, and the direction it was facing, and the hunger level. In all cases, we used some bigger tile chunks and some smaller (in terms of x and z). For example, in the 9x9 dungeon, for the gold block and chicken killing, we used tiles whose dimensions were $\{3, 3, 1\}$ for x, z, and direction, as well as tiles with dimensions $\{2, 2, 1\}$ and $\{1, 1, 1\}$. We also used multiple tilings for each set of dimensions with random starting coordinates to introduce jitter, so we had overlapping tilings. Using overlapping tilings, as well as having bigger and smaller tiles, gave the algorithm general information about certain areas of the dungeon as well as more specific information from nearby tiles.

After training with this algorithm just a few times, the agent has enough information to approximate the best possible action path immediately when started. After a few training iterations, the time the agent took to find the gold block approached the time it took using astar, which gives the agent all the necessary information and requires no guessing at all. This tells us that to the agent, finding the best possible path is essentially a solved problem after giving it just a few minutes' worth of feedback (in the 9x9 and 3x3 environments, anyway). We have no astar to compare to when killing the chicken, but the time the agent takes goes down significantly after multiple trainings although as may be expected, teaching the agent to follow and kill a chicken takes much longer than finding a gold block to begin with.

As for hunger exploration, it turned out that this type of CMAC tile coding was actually not good enough. The hunger

was not weighted enough relative to the other variables. The agent would go out and come back, but not as a function of hunger, and always following the same path. Moreover, it was a lot harder just to train this behavior given the extra variable to tile on. (Adding the extra dimension just confused things.) What we really needed was to train an entirely new policy once the agent reached half hunger (to get optimal risky exploration).

We used two different methods to provide the agent with user feedback. First, we set up key input handlers that are called when two buttons are pressed - in this case, O provided positive feedback, and P provided negative feedback. This method was very consistent and effective, as we expected.

After getting the agent to go to a gold block and kill a chicken with good and bad buttons, we started experimenting with using emotion detection on images of the users face while they watch the agent complete a task to provide positive and negative feedback, depending on the valence of the person's face. In other words, while the user watches the agent try to achieve a goal, if the user looks happy, the agent will receive positive feedback, to associate with the current action and state. If the user looks angry, the agent will receive negative feedback. Thus, the agent learns in a similar way to using the good and bad buttons, but without requiring the user to touch the computer or engage in any way other than watching. We first tried Indico.io's emotion detection, but it didnt give us any useful data. We ended up using OpenCV's face classification algorithm by training it on three different classifications (happy, neutral, and angry - examples shown in Fig. 2). The classification the picture of the user matched with was considered the valence of their emotion (-1, 0, or 1), and given as feedback to the agent. OpenCV provides three classifiers: Fisher, EigenFace, and LBPH. We tried all three and determined that LBPH does the best consistent job at differentiating the three classifications for our purposes.

Fig. 2.

III. UPPER LIMITS OF CURRENT PRACTICE

I-Learning is the tool we will be using to accomplish our tasks. It has not been used in BurlapCraft before. Additionally, emotion detection has not been used in BurlapCraft. Nor do we know of hunger based exploration in BurlapCraft.

IV. WHAT'S NEW IN OUR APPROACH

We are the first to use I-Learning in BurlapCraft. Our work proves that one can teach agents to do nontrivial things in a gridworld like Minecraft using simple methods of feedback. Also, it gives us more insight into the capabilities and limitations of I-Learning with CMAC tile coding.

Furthermore, the fact that we were able to use emotion detection on the users face with relatively little training time for the classifiers has implications for the way people can interact with agents. With physical robots, as they get better at communicating more naturally with humans, we will want them to read our facial expressions, and maybe even the tone of our speech, to tell how happy we are with them.

V. WHO CARES

The research we have done with I-Learning in Burlap Craft is the first combination of the two. We have gained insight into capabilities and limitations of I-Learning with CMAC tile coding and made it easier to do future research with the two. Minecraft is a useful environment to test I-Learning in given that sensory information and control of the environment are completely accessible. Moreover, the kinds of tasks we have tackled in Minecraft have analogs in the real world. Finding a gold block is analogous to learning a shortest-path navigation. Killing the chicken is analogous to target following. And hunger exploration is analogous to a robot trying to explore complete tasks before its battery drains.

The research we have done with emotion detection also has significant implications. Having a method of communicating to a robot how good the state of the world is at that moment that doesn't require physical interaction with the robot has many applications: it adds a lot of information relatively easily and quickly. Adding this dimension can be used to do things like narrow the search space of language comprehension. Humans, when they communicate, take many environmental cues into account, facial expression primary among them.

One specific application is elder care. There already exist robots whose job is to provide services to older people living alone. Using the ideas we have combined in our research, we could make a robot that checks up on its owner, and can tell if they are in pain or require help in some way just by looking at their face. Overall, the idea of combining emotion detection with learning algorithms means that we will all soon be able to interact more naturally and effortlessly with AI agents in our environment.

VI. FUTURE RESEARCH

For I-Learning and CMAC tiling, more needs to be tested to determine when uniform tiling is better than the random jitter tiling we used. Perhaps there is a tiling that makes hunger exploration doable with I-Learning and CMAC tiling. (We started to look into this, and it seemed unlikely, but more experimentation is needed to make conclusions.)

As for emotion detection, the neutral face detection is often confused with anger. This may just be the face we tested on. More experimentation is needed here. Additionally,

combinations of LBPH and Fisher where a weighted confidence of each is used to determine which to use may be more accurate than either alone. (We tried one unweighted trial and got a very low success rate, but more experimentation is needed to make conclusions.) Furthermore, deep learning may be able to greatly improve emotion detection.

VII. DATA

A. Gold block - 3x3

Algorithm	Avg time (s)	Trials
astar	6.32	3
rmax	36.28	3
I-Learning untrained (good/bad buttons)	19.62	3
I-Learning trained (good/bad buttons)	6.98	3
I-Learning untrained (emotion detection)	19.77	2
I-Learning trained (emotion detection)	5.59	2

B. Gold block - 9x9

Algorithm	Avg time	Trials
astar	26.93	3
rmax	$>$ 4:00.00	3
I-Learning untrained (good/bad buttons)	57.43	3
I-Learning trained (good/bad buttons)	26.36	3

C. Killing a chicken - 3x3

Algorithm	Avg time	Trials
I-Learning untrained (good/bad buttons)	3:09.16	2
I-Learning trained (good/bad buttons)	10.95	2

D. Emotion detection training – 3x3

Trial	Test A
1	34.71
2	30.76
3	12.17
4	17.49
5	42.13
6	20.09
7	54.56
8	53.50
9	13.84
10	41.14
11	24.71
12	24.37
13	23.66
14	12.74
15	22.47
16	1:04.29 (got stuck, then optimal path)
17	7.98
18	4.26 (optimal path)
19	5.21 (optimal path)

Trial	Test B
1	4.83 (optimal path - lucky)
2	6.30 (optimal path)
3	5.97 (optimal path)

E. Comparing OpenCV face classifiers

Retrained for each trial on the same face, position, and environment. 60 photos per emotion for both the training set and each of the 3 detection trials. The table below contains the number of photos in each trial the given algorithm correctly identified (out of 60).

Trial	**Fisher**	**Eigen**	**LBPH**
1 - Angry	0	58	60
1 - Neutral	60	1	6
1 - Happy	60	3	60
2 - Angry	60	60	60
2 - Neutral	0	0	0
2 - Happy	0	0	0
3 - Angry	60	60	34
3 - Neutral	60	58	58
3 - Happy	60	60	60

Eigen is the consistently the worst as predicting every emotion. Fisher consistently better at predicting Neutral compared to LBPH, but worse for happy and angry.

F. Comparing OpenCV face classifiers overall

Face classifier	**Success Rate**
Fisher	0.667
Eigen	0.556
LBPH	0.796